

Reviewer Pack — Minimal Rank of Non-Archimedean Valuations over Boolean Circ...

Entry 1dc9618b9b6d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 14:26:34 UTC

Minimal Rank of Non-Archimedean Valuations over Boolean Circuit Satisfiability

Entry ID: 1dc9618b9b6d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 14:26:34 UTC

1. Conjecture

Field A (mathematical branch): Non-Archimedean Analysis **Field B** (complexity object): Boolean Circuit Satisfiability

Statement:

[‘For every boolean circuit C with n inputs, the minimal rank of the non-archimedean valuation V_C associated with C is $\Theta(\log n)$.’, ‘This valuation assigns to each variable in C and its negation a value from an ordered field extension of the real numbers, such that the circuit can be evaluated as a function on this field.’, ‘The statement holds for all instances with up to 40 variables, providing a quantitative relation between non-archimedean valuations and circuit satisfiability.’]

Rationale (proposer’s reasoning):

[‘Non-archimedean analysis provides a framework to study the behavior of functions over ordered fields that extend the real numbers. By associating such valuations with boolean circuits, we can potentially expose new structural properties that are not apparent in classical models.’, ‘If this conjecture is true, it would suggest a deep connection between non-archimedean analysis and complexity theory, providing insights into the nature of boolean circuit satisfiability.’, ‘The minimal rank of a valuation could serve as an interesting complexity measure for boolean circuits, complementing existing measures like size or depth.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 703423c5b2155b4d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average minimal rank of non-archimedean valuations over all generated circuits is within 1 standard deviation of $\Theta(\log n)$ for at least 25 out of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "non-archimedean valuation" AND "Boolean circuit satisfiability" - "minimal rank" AND "non-archimedean valuation" AND " $\Theta(\log n)$ " - "ordered field extension" AND "Boolean circuit evaluation" AND "non-archimedean analysis"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return 'x0'
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return f'({left} OR {right})'

    def evaluate_circuit(circuit, valuation):
        if circuit.startswith('(') and circuit.endswith(')'):
            circuit = circuit[1:-1]
        if ' OR ' in circuit:
            left, right = circuit.split(' OR ')
            return evaluate_circuit(left, valuation) or evaluate_circuit(right, valuation)
        elif ' AND ' in circuit:
            left, right = circuit.split(' AND ')
            return evaluate_circuit(left, valuation) and evaluate_circuit(right, valuation)
        else:
            var, negated = circuit.startswith('¬'), circuit[1:] if circuit.startswith('¬') else circuit
            return valuation[var] != negated

    def non_archimedean_valuation(circuit):
        if circuit == 'x0':
            return {'x0': 0}
```

```

elif ' OR ' in circuit:
    left, right = circuit.split(' OR ')
    val_left = non_archimedean_valuation(left)
    val_right = non_archimedean_valuation(right)
    for var in set(val_left) | set(val_right):
        if var not in val_left or var not in val_right:
            return {var: 1}
    return {var: max(val_left[var], val_right[var]) for var in val_left}
elif ' AND ' in circuit:
    left, right = circuit.split(' AND ')
    val_left = non_archimedean_valuation(left)
    val_right = non_archimedean_valuation(right)
    for var in set(val_left) | set(val_right):
        if var not in val_left or var not in val_right:
            return {var: 0}
    return {var: min(val_left[var], val_right[var]) for var in val_left}
else:
    var, negated = circuit.startswith('~'), circuit[1:] if circuit.startswith('~') else circuit
    return {var: int(negated)}

def minimal_rank(valuation):
    rank = 0
    for value in valuation.values():
        rank = max(rank, math.ceil(math.log2(value + 1)))
    return rank

n = random.randint(5, 40)
circuit = generate_circuit(n)
valuation = non_archimedean_valuation(circuit)
rank = minimal_rank(valuation)

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank == math.log2(n) + 1,
    "counterexample": "" if rank == math.log2(n) + 1 else f"Counterexample for n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Counterexample found\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45884b6e.py", line 90, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45884b6e.py", line 73, in run_trial
    valuation = non_archimedean_valuation(circuit)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45884b6e.py", line 46, in non_archimedean_valuation
    left, right = circuit.split(' OR ')
                  ~~~~~^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11911
2	preregistration	ollama_remote	glm4:latest	0	0	6031
3	novelty	ollama_remote	glm4:latest	0	0	4874
4	novelty	ollama_remote	glm4:latest	0	0	6339
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45903
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9714
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10934
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11238
9	judge	ollama_remote	glm4:latest	0	0	8133

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115077 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1dc9618b9b6d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1dc9618b9b6d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1dc9618b9b6d.tar.gz (if generated)